



PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

Jackfruit Phase-1

Software Requirements Specification for AttendEase – Student Attendance Management System

Item	Details
Version	1.0 (Approved)
Prepared by	Team 3 – Anshdeep Singh Sachdeva, Arundhati K, Atharva Ashish Vyas, Bhoomika Dayanand Jituri
Organization	PES University, Bangalore – Dept. of CSE
Course	UE24CS341A – Software Engineering
Date Created	02 September 2026

1. Introduction

1.1 Purpose

This document specifies the software requirements for Version 1.0 of AttendEase, a web-based Student Attendance Management System being developed by Team 3 as the UE24CS341A Software Engineering mini-project. It covers the complete scope of the system as proposed in the project synopsis: authentication, course/timetable administration, attendance marking, student self-service tracking, correction workflow, and reporting.

1.2 Intended Audience and Reading Suggestions

This SRS is intended for: (a) the development team, who will use Sections 3–7 as the basis for architecture, design and test-case development; (b) the course instructor/evaluator, who will use it to assess requirement quality and later trace it to design, code and tests; and (c) any future maintainer of the system. Readers should review Sections 1–2 for context, Section 5 for the detailed functional requirements, and Section 6 for quality expectations.

1.3 Product Scope

AttendEase digitizes the department's attendance workflow. Administrators configure courses, sections and timetables; faculty mark attendance per class session; students track attendance in real time and raise corrections; and the system automatically computes attendance percentages and flags defaulters. The full in-scope/out-of-scope breakdown is given in Section 2.3.

1.4 References

- PES University, Dept. of CSE – “Jackfruit – Software Engineering Project Guidelines 2026” (UE24CS341A).
- PES University – Software Engineering Level 3 Mini-Project Student Submission Guide (Coffee Kiosk exemplar).
- Team 3 Synopsis / Project Proposal, AttendEase, v1.0.
- IEEE SRS template (as adapted in the PES-supplied SRS template).

2. Overall Description

2.1 Product Perspective

AttendEase is a new, self-contained product; it is not a follow-on to an existing departmental system and does not replace any commercial software currently in use. It is designed as a standalone three-tier web application (browser client, Django application server, relational database) that could later integrate with a university ERP, though that integration is explicitly out of scope for this project.

2.2 Product Functions

- User registration and role-based login for Student, Faculty and Administrator.
- Course, section, timetable and faculty-subject-assignment management.
- Attendance session creation and per-student marking by faculty.
- Student self-service attendance dashboard (subject-wise and cumulative).
- Attendance correction request and faculty approval/rejection workflow.
- Automatic attendance-percentage computation and defaulter identification.
- Consolidated and defaulter report generation/export.
- Administrator operational dashboard and user-account management.

2.3 User Classes and Characteristics

User Class	Privileges	Characteristics
Student	Register/login, view own attendance and history, raise correction requests	Largest user base; non-technical; frequent, short sessions
Faculty	Mark/edit attendance for assigned sessions, approve/reject correction requests	Moderate technical comfort; time-constrained during class hours

User Class	Privileges	Characteristics
Administrator	Manage courses/sections/timetables/faculty assignment, manage user accounts, configure threshold, view reports	Small user base; higher privilege; departmental office staff

2.4 Operating Environment

- Server: Linux (Ubuntu 22.04 or later), Python 3.11+, Django 4.x / Django REST Framework.
- Database: PostgreSQL 14+ (MySQL 8+ as an alternative).
- Client: any evergreen desktop or mobile browser (Chrome, Firefox, Edge – latest two versions), responsive layout via Bootstrap 5.
- Deployment: containerized with Docker; CI/CD via GitHub Actions.

2.5 Design and Implementation Constraints

- Development must follow Agile methodology with visible sprints, backlog and user stories on GitHub Projects/Jira.
- Git/GitHub must be used for version control, with meaningful commit history from every team member.
- A CI/CD pipeline, code reviews and security validation are mandatory before code is merged.
- The project must be completed within one academic semester by a 4-member team.
- A relational database must be used to preserve referential integrity between students, courses, sessions and attendance records.

2.6 Assumptions and Dependencies

- Every student and faculty member has a valid institutional email address for account registration.
- Course, section and timetable master data is available/entered by the administrator before the semester begins.
- Users access the system over a stable internet connection; offline attendance capture is not supported.
- The system depends on the continued availability and API stability of Django, DRF, and the chosen database engine.
- Payment/biometric hardware integration is not available and is therefore out of scope.

3. External Interface Requirements

3.1 User Interfaces

The system exposes a responsive web UI with three role-specific views: a Student dashboard (attendance summary, history, correction request form), a Faculty console (session creation, roster marking, correction approvals), and an Admin console (course/section/timetable/user management, reports). All screens share a common header/navigation, consistent button placement (Save/Submit/Cancel), and inline, field-level error messages. Detailed wireframes will be documented in the separate Design phase submission.

3.2 Software Interfaces

The Django application communicates with PostgreSQL via the Django ORM for all persistent data (users, courses, sections, sessions, attendance records, correction requests). Internal AJAX calls from the browser use a JSON REST API exposed via Django REST Framework. No third-party commercial software components are integrated in v1.0.

3.3 Communications Interfaces

All client-server communication occurs over HTTPS. Outbound email (e.g., correction-request notifications to faculty, low-attendance alerts to students) is sent via SMTP through the institution's mail relay or a transactional email service. No other communication protocols are used in this version.

4. Analysis Models

4.1 Use Cases

UC ID	Use Case	Primary Actor	Precondition	Outcome
UC-01	Register / Login	Student / Faculty / Admin	Application available	Authenticated, role-based session
UC-02	Manage Course & Timetable	Administrator	Admin authenticated	Course/section/timetable created or updated
UC-03	Assign Faculty to Subject-Section	Administrator	Course & section exist	Faculty assignment saved
UC-04	Start & Mark Attendance Session	Faculty	Faculty authenticated; session scheduled	Attendance session submitted
UC-05	View Attendance Dashboard	Student	Student authenticated	Subject-wise & cumulative % displayed
UC-06	Raise Correction Request	Student	Attendance session exists	Correction request created
UC-07	Approve/Reject Correction Request	Faculty	Pending correction request exists	Attendance record updated or request closed
UC-08	Generate Defaulter / Attendance Report	Faculty / Administrator	Attendance data exists	Report generated / exported

4.2 Domain / Entity Model (Summary)

Key entities and relationships (a full ER diagram will be produced in the Design-phase submission): a Course has many Sections; a Section has many enrolled Students and a weekly Timetable of Sessions; each Session belongs to one Section-Subject-Faculty combination and produces many AttendanceRecords, one per enrolled Student; a Student may raise a CorrectionRequest against one AttendanceRecord, which a Faculty member resolves. A User (Student/Faculty/Admin) is the base identity/authentication entity referenced by Student and Faculty.

5. System Features

Features are grouped into the four ownership roles the team has divided the project into. Each role is owned end-to-end (requirements → design → implementation → testing) by one team member; role assignment is given in the Synopsis, Section 8.

5.1 Role 1 — User Access & Student Portal

5.1.1 Authentication & Role-Based Access

Description and Priority: Allows a Student, Faculty member or Administrator to register, log in, and be administered by role. *Priority:* High.

Stimulus/Response: User submits registration or login credentials → system validates them, creates the account or session, and routes the user to their role-specific dashboard.

Req ID	Functional Requirement	Priority	Verification
FR-001	The system shall allow a new user to register using their institutional email and a selected role (Student or Faculty).	High	Functional test
FR-002	The system shall allow a registered user to log in using their email and password.	High	Functional test
FR-003	The system shall issue a role-based session upon successful login, granting access only to that role's screens.	High	Authorization test
FR-004	The system shall lock a user account for 15 minutes after 5 consecutive failed login attempts.	Medium	Negative test

Req ID	Functional Requirement	Priority	Verification
FR-005	Administrator accounts shall be created only by an existing administrator, not through self-registration.	High	Functional test
FR-034	The system shall allow an administrator to add, edit or deactivate a user account.	High	Functional test

5.1.2 Student Attendance Dashboard

Description and Priority: Gives a student real-time, self-service visibility into their attendance. Priority: High.

Req ID	Functional Requirement	Priority	Verification
FR-020	The system shall display to a logged-in student their attendance percentage per enrolled subject.	High	Functional test
FR-021	The system shall display to a logged-in student their overall cumulative attendance percentage.	High	Functional test
FR-022	The system shall display a session-wise attendance history (date, subject, status) to the student.	Medium	Functional test
FR-023	The system shall visually flag any subject where the student's attendance is below 75%.	Medium	Functional test

5.2 Role 2 — Academic & Attendance Setup

5.2.1 Student & Academic Management

Description and Priority: Allows an administrator to define courses, sections, timetable slots, and faculty-subject-section assignment. Priority: High.

Stimulus/Response: Administrator submits course/section/timetable/assignment data → system validates for conflicts and persists the configuration.

Req ID	Functional Requirement	Priority	Verification
FR-006	The system shall allow an administrator to create a course with a unique course code.	High	Functional test
FR-007	The system shall allow an administrator to create one or more sections/batches under a course.	High	Functional test
FR-008	The system shall allow an administrator to define a weekly timetable slot for a section-subject-faculty combination.	High	Functional test
FR-009	The system shall reject creation of two timetable slots with overlapping time for the same faculty member.	High	Negative test
FR-010	The system shall allow an administrator to enroll students into a section.	High	Functional test
FR-011	The system shall allow an administrator to assign a faculty member to a subject-section pair.	High	Functional test
FR-012	The system shall display to a logged-in faculty member only the subjects/sections assigned to them.	High	Authorization test
FR-013	The system shall allow an administrator to reassign a faculty member to a different section.	Medium	Functional test

5.2.2 Attendance Recording

Description and Priority: Allows a faculty member to open an attendance session for a scheduled class and mark each enrolled student present or absent. Priority: High.

Stimulus/Response: Faculty starts a session for a scheduled slot → system displays the enrolled roster → faculty marks each student and submits → system persists the attendance record and triggers percentage recalculation.

Req ID	Functional Requirement	Priority	Verification
FR-014	The system shall allow a faculty member to start an attendance session for a scheduled class slot assigned to them.	High	Functional test
FR-015	The system shall display the enrolled student roster for the selected session.	High	Functional test
FR-016	The system shall allow a faculty member to mark each student in the roster as Present or Absent.	High	Functional test
FR-017	The system shall allow a session to be submitted only after every enrolled student has a marked status.	High	Negative + positive test
FR-018	The system shall reject creation of a duplicate attendance session for the same section-subject-date-slot.	High	Negative test
FR-019	The system shall allow a faculty member to edit a submitted session's attendance within 24 hours of submission.	Medium	Functional test

5.3 Role 3 — Attendance Updates & Corrections

5.3.1 Attendance Management

Description and Priority: Governs how already-recorded attendance is viewed and subsequently changed, once initial marking (Section 5.2.2) is complete. Priority: Medium.

Req ID	Functional Requirement	Priority	Verification
FR-037	The system shall allow a faculty member to search and view attendance records for any session they own, filterable by date range.	Medium	Functional test
FR-038	The system shall prevent any edit to a submitted attendance record after 24 hours except through an approved correction request.	High	Negative test

5.3.2 Attendance Correction Requests

Description and Priority: Provides a controlled channel for correcting attendance mistakes without letting students edit records directly. Priority: Medium.

Req ID	Functional Requirement	Priority	Verification
FR-024	The system shall allow a student to raise a correction request for a specific session, with a reason.	Medium	Functional test
FR-025	The system shall notify the concerned faculty member of a new, pending correction request.	Medium	Integration test
FR-026	The system shall allow the faculty member who owns the session to approve or reject the request.	Medium	Authorization test
FR-027	The system shall update the attendance record only after the corresponding request is approved.	High	Functional test
FR-028	The system shall maintain an audit log of every correction request and its outcome.	Medium	System test

5.4 Role 4 — Attendance Analytics & Reporting

5.4.1 Attendance Shortage Identification

Description and Priority: Computes attendance percentages and surfaces students who fall below the configured minimum. Priority: High.

Req ID	Functional Requirement	Priority	Verification
FR-029	The system shall recompute each student's attendance percentage per subject after every session submission.	High	Unit + system test

Req ID	Functional Requirement	Priority	Verification
FR-030	The system shall generate a defaulter list of students below the configured minimum threshold (default 75%) per subject.	High	Functional test
FR-035	The system shall allow an administrator to configure the minimum attendance threshold used for defaulter detection.	Medium	Functional test

5.4.2 Attendance Reports

Description and Priority: Gives faculty and administrators consolidated, exportable, department-wide reporting. Priority: Medium.

Req ID	Functional Requirement	Priority	Verification
FR-031	The system shall allow faculty/administrators to export the defaulter list and consolidated attendance report as CSV or PDF.	Medium	Functional test
FR-032	The system shall allow reports to be filtered by course, section, subject and date range.	Medium	Functional test
FR-033	The system shall display system-wide statistics (total students, faculty, courses, average attendance) to the administrator.	Medium	Functional test
FR-036	The system shall log all administrative actions for audit purposes.	Medium	System test

6. Other Nonfunctional Requirements

6.1 Performance Requirements

ID	Measurable Requirement	Verification
NFR-001	95% of attendance-marking submissions shall complete within 2 seconds under a load of up to 60 concurrent users.	Performance test
NFR-002	The student dashboard shall load the attendance summary within 3 seconds under normal load.	Performance test

6.2 Safety Requirements

ID	Measurable Requirement	Verification
NFR-003	The system shall retain attendance records for a minimum of 3 academic years without data loss.	Operational review
NFR-004	The system shall perform an automated daily database backup.	Configuration review

6.3 Security Requirements

ID	Security Requirement	Validation Approach
SEC-001	Passwords shall be stored using a salted hash (bcrypt/argon2); never in plaintext.	Code/configuration review
SEC-002	Role-based access control shall prevent students from accessing faculty or admin endpoints.	Authorization tests
SEC-003	The system shall validate and sanitize all user-supplied input to prevent SQL injection and XSS.	Negative / security tests
SEC-004	A session shall expire automatically after 30 minutes of inactivity.	Security test
SEC-005	All client-server communication shall occur over HTTPS.	Configuration review
SEC-006	Failed logins, correction-request decisions and administrative actions shall be logged with timestamp and user ID.	System test

6.4 Software Quality Attributes

ID	Category	Measurable Requirement
NFR-005	Maintainability	Business logic shall be separated into modular Django apps with unit-test coverage of at least 70%.
NFR-006	Usability	A faculty member shall be able to mark attendance for a 60-student class in under 3 minutes.
NFR-007	Availability	The system shall be available at least 99% of scheduled class hours (8 AM–6 PM, Mon–Sat), excluding planned maintenance.
NFR-008	Portability	The web application shall render correctly on the latest two versions of Chrome, Firefox and Edge.

6.5 Business Rules / Domain Rules

- Only an enrolled student can be marked present or absent for a given section-subject session.
- A faculty member can mark attendance only for sessions/sections assigned to them.
- A submitted attendance session is locked from direct editing after 24 hours; only an approved correction request can change it thereafter.
- A student below the configured minimum threshold is flagged as a defaulter for that subject.
- Only an administrator can create/deactivate user accounts and configure the attendance threshold.

- A correction request may be approved or rejected only by the faculty member who owns the corresponding session.

7. Other Requirements

- Database requirement: a relational database (PostgreSQL/MySQL) enforcing referential integrity between Users, Courses, Sections, Sessions, AttendanceRecords and CorrectionRequests (see Appendix B for field layouts).
- Legal/Institutional requirement: attendance data is an institutional record; access is restricted to authorized roles and retained per Section 6.2.
- Reuse objective: the team will reuse Django's built-in authentication framework, extended with a custom Role field, rather than building authentication from scratch.

Appendix A: Glossary

Term	Definition
Attendance Session	A single scheduled class instance (subject + section + date + slot) for which attendance is recorded.
Defaulter	A student whose attendance percentage in a subject is below the configured minimum threshold.
Correction Request	A student-initiated request to amend a previously submitted attendance record.
RBAC	Role-Based Access Control – restricting system functions based on the logged-in user's role.
Section / Batch	A group of students following a common timetable within a course and semester.
USN / SRN	University Seat Number / Student Registration Number – unique student identifiers.
Threshold	The configurable minimum attendance percentage (default 75%) below which a student is a defaulter.

Appendix B: Field Layouts

B.1 Core Data Fields

Field	Length	Data Type	Description	Mandatory
user_id	36	UUID/String	Unique identifier for any system user	Y
email	100	String	Institutional login email (unique)	Y
password_hash	255	String	Salted hash of the user's password	Y
role	20	Enum	Student / Faculty / Admin	Y
usn	15	String	University Seat Number (students)	Y
srn	15	String	Student Registration Number (students)	Y
course_id	36	UUID/String	Unique course identifier	Y
course_code	15	String	Course code, e.g., UE24CS341A	Y
section_id	36	UUID/String	Unique section/batch identifier	Y
session_id	36	UUID/String	Unique attendance session identifier	Y
session_date	8	Date	Date of the class session	Y
status	10	Enum	Present / Absent for a student in a session	Y
threshold_pct	5	Numeric	Configured minimum attendance percentage	N
correction_reason	255	String	Student-supplied reason for a correction request	Y (when requested)

B.2 Sample Report Requirements

Student Attendance Report	Session Report
USN / SRN / Student Name	Session ID
Course / Section / Subject	Subject / Section / Faculty
Total Sessions Conducted	Date
Sessions Present	Total Enrolled
Attendance %	Present Count / Absent Count
Status (OK / Defaulter)	Submission Timestamp

Appendix C: Requirement Traceability Matrix

Architecture, Design, Code and Test-Case references will be populated during the Design and Implementation phases. Representative rows are shown below, one per role; the full matrix will cover all 38 functional requirements and all security/nonfunctional requirements.

Req ID	Brief Description	Owning Role	Architecture Ref.	Design Ref.	Code File Ref.	Test Case ID
FR-001	User registration with role	Role 1 – User Access & Student Portal	TBD	TBD	TBD	TBD
FR-020	Student subject-wise %	Role 1 – User Access & Student Portal	TBD	TBD	TBD	TBD
FR-006	Create course with unique code	Role 2 – Academic & Attendance Setup	TBD	TBD	TBD	TBD
FR-014	Start attendance session	Role 2 – Academic & Attendance Setup	TBD	TBD	TBD	TBD
FR-038	Lock record after 24h except via correction	Role 3 – Attendance Updates & Corrections	TBD	TBD	TBD	TBD
FR-024	Raise correction request	Role 3 – Attendance Updates & Corrections	TBD	TBD	TBD	TBD
FR-030	Generate defaulter list	Role 4 – Attendance Analytics & Reporting	TBD	TBD	TBD	TBD
FR-031	Export consolidated report	Role 4 – Attendance Analytics & Reporting	TBD	TBD	TBD	TBD
SEC-002	RBAC for endpoints	Role 1 – User Access & Student Portal	TBD	TBD	TBD	TBD
NFR-001	2s response for marking	Role 2 – Academic & Attendance Setup	TBD	TBD	TBD	TBD

Revision History

Name	Date	Reason for Changes	Version
Team 3	02-Sep-2026	Initial draft for Requirements-phase submission	1.0
Team 3	03-Sep-2026	Regrouped System Features into 4 team ownership roles; added FR-037/FR-038	1.1